FastAPI PDF Loading Error - da81855af

USER

I am using the fastapi as my backend engine in python, but when I pass the path of a pdf file to display it on my HTML page the js code gave me that error:
pdf.min.js:22 Not allowed to load local resource: file:///D:/Main_Project/users/bf6d9018-8ef2-47fc-a54d-49f2e5b36cf9_Taha/17475688466044_test19.pdf
request @ pdf.min.js:22
requestFull @ pdf.min.js:22
PDFNetworkStreamFullRequestReader @ pdf.min.js:22
getFullReader @ pdf.min.js:22
(anonymous) @ pdf.min.js:22
(anonymous) @ pdf.min.js:22
_createStreamSink @ pdf.min.js:22
MessageHandler._onComObjOnMessage @ pdf.min.js:22
document:1267 Error processing PDF: MissingPDFException {message: 'Missing PDF "file:///D:/Main_Project/users/bf6d901…54d-49f2e5b36cf9_Taha/17475688466044_test19.pdf".', name: 'MissingPDFException', stack: 'Error\n   at BaseExceptionClosure (https://cdnjs.c…e.com/ajax/libs/pdf.js/2.11.338/pdf.min.js:22:294'}

BOT

This error occurs because browsers **do not allow JavaScript to load local files** (using `file:///` URLs) for security reasons. When you pass a local file path (like `file:///D:/...`) to your frontend, the browser tries to fetch it directly from your filesystem, which is **blocked**.

How to Fix

You need to **serve the PDF file through your FastAPI backend** as a downloadable or viewable resource, and then access it via an HTTP(S) URL from your frontend.

1. Serve the PDF file with FastAPI

Suppose your PDF is at `D:/Main_Project/users/bf6d9018-8ef2-47fc-a54d-49f2e5b36cf9_Taha/17475688466044_test19.pdf`. You can create an endpoint to serve it:

```
from fastapi import FastAPI
from fastapi.responses import FileResponse

app = FastAPI()

@app.get("/pdf/{filename}")
def get_pdf(filename: str):
```

```
    file_path = f"D:/Main_Project/users/bf6d9018-8ef2-47fc-a54d-49f2e5b36cf9_Taha/{filename}"
    return FileResponse(path=file_path, media_type='application/pdf', filename=filename)
```

Now, your PDF is accessible at:
```
http://localhost:8000/pdf/17475688466044_test19.pdf
```

2. Update your frontend to use the HTTP URL

Instead of passing a `file:///` path, pass the HTTP URL to your JS/pdf.js code:

```js
const url = "http://localhost:8000/pdf/17475688466044_test19.pdf";
// Use this URL with PDF.js
```

3. Security Note

- **Never** expose arbitrary file paths. Always validate or sanitize the filename, or use a database to map allowed files.
- For production, use a proper static files setup or authentication as needed.

Summary:
You cannot load local files directly in the browser for security reasons. Always serve files via HTTP from your backend, and use those URLs in your frontend code.

If you need a more complete example or help with pdf.js integration, let me know!

USER

this is the call from the fastapi backend:

```
    return ht_pages.TemplateResponse(
        "document.html",
        {
            "request":req,
            "name":full_name,
            "file_path":file_path,
            "output":json_data
            } # If you have any additional parameters you can add them in this dictionary
    )
```

and this is the receiving code from js:

```
const pdfPath = "{{ file_path | replace('\\', '\\\\') }}";
```

now tell me what to do exactly

BOT

Great, you're using FastAPI with Jinja2 templates and passing `file_path` to your HTML/JS. The problem is that `file_path` is a **local file path** (e.g., `D:/...`), which browsers cannot access directly. You need to **serve the PDF via FastAPI** and pass an HTTP URL to your frontend.

Here's exactly what you should do:

---

1. **Add a FastAPI Route to Serve the PDF**

Add this endpoint to your FastAPI app (adjust the path logic as needed):

```
from fastapi.responses import FileResponse
from fastapi import HTTPException
import os

@app.get("/pdf/{filename}")
def get_pdf(filename: str):
    # Build the full path securely
    base_dir = "D:/Main_Project/users/bf6d9018-8ef2-47fc-a54d-49f2e5b36cf9_Taha"
    file_path = os.path.join(base_dir, filename)
    if not os.path.isfile(file_path):
        raise HTTPException(status_code=404, detail="File not found")
    return FileResponse(path=file_path, media_type='application/pdf', filename=filename)
```

---

2. **Pass the HTTP URL to Your Template**

Instead of passing the local file path, pass the **HTTP URL** to your template. For example:

```
filename = os.path.basename(file_path)
pdf_url = f"/pdf/{filename}"

return ht_pages.TemplateResponse(
    "document.html",
    {
        "request": req,
        "name": full_name,
        "file_path": pdf_url,  # Pass the HTTP URL, not the local path
        "output": json_data
    }
)
```

---

3. **Update Your JS to Use the HTTP URL**

In your template, use the `file_path` variable as the URL:

```
```js
const pdfPath = "{{ file_path }}"; // This will now be something like "/pdf/
17475688466044_test19.pdf"
```
```

```

If your FastAPI is running on a different port or domain, prepend the full URL:

```js
const pdfPath = "http://localhost:8000{{ file_path }}";
```

---

4. **Use `pdfPath` with PDF.js**

Now, use `pdfPath` as the source for PDF.js or your viewer.

Summary of Changes

- **Backend:** Serve the PDF via an HTTP endpoint.
- **Template:** Pass the HTTP URL (not the local file path) to the frontend.
- **Frontend:** Use the HTTP URL in your JS code.

If you follow these steps, your browser will be able to fetch and display the PDF.
Let me know if you need a full working example or help with PDF.js integration!
```